

Projekt Bericht inclusive Project
Requirements Specification
Home Security System

Versionsverlauf

Version	Status	Datum	Verantwortliche*r	Änderungsgrund
V0.1	initialisiert	09.02.2026	Marius Marik	Erstellung und Erstbearbeitung des PRS-Dokuments um Grundlagen des Projekts festzuhalten.
V1.0	Abgeschlossen	16.06.2026	Marius Marik	Finale Dokumentation

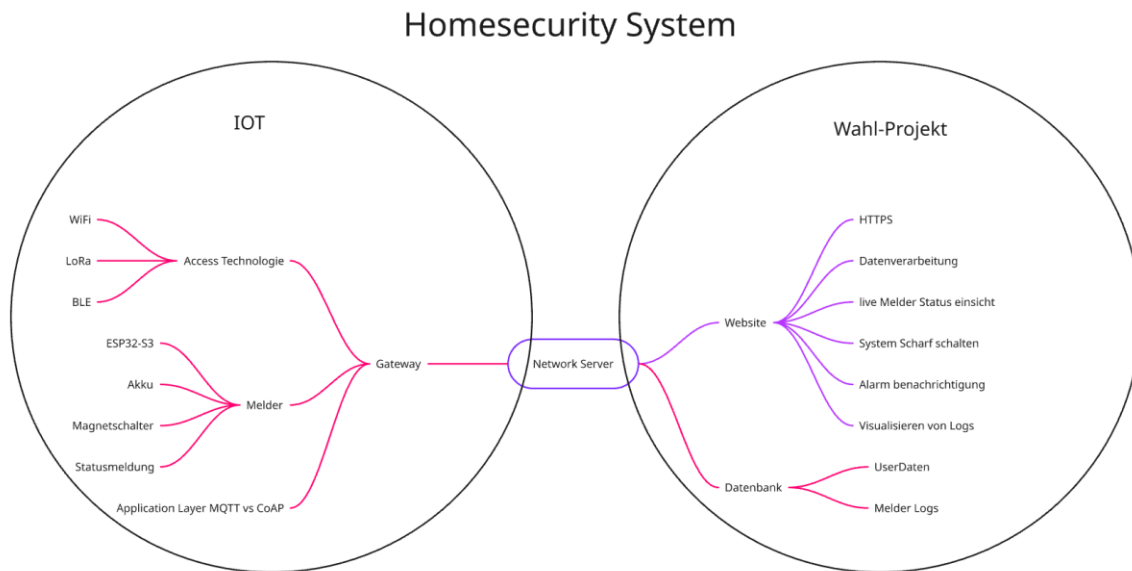
Abkürzungen

MVP Minimum Viable Product (Erstentwicklung mit wichtigsten Funktionen)

Inhalt

1. Homesecurity System – Marius Marik	4
1.1. Einleitung	4
1.2. Überblick	5
1.3. Functional Requirements (Software)	5
1.3.1. Soll-Kriterien (S)	6
1.3.2. Könnte-Kriterien (C)	6
1.3.3. Abgrenzungs-Kriterien (W)	6
1.3.4. Muss-Kriterien (M) Non Functional	6
1.4. Functional Requirements (Hardware)	7
1.4.1. Muss-Kriterien (M)	7
1.4.2. Soll-Kriterien (S)	7
1.4.3. Kann Kriterien	7
1.5. Non-functional Requirements (Hardware)	7
1.6. Technology Stack	8
1.7. Netzwerkplan	9
1.8. Schaltplan	9
1.9. Sicherheitsmaßnahmen	11
1.10. Configuration Screenshots	13
1.11. Reflexion	17
1.12. Outcome	18
1.13. Zeitaufwand	19
1.14. GitHub Repository	21

1.Homesecurity System – Marius Marik



1.1.Einleitung

Das Projekt Homesecurity System soll dazu dienen meine bestehende Alarmanlagen Lösung, welche deutliche Schwachstellen aufzeigt, abzulösen.

Die Derzeitige Alarmanlage besteht aus einer "Zentrale", welche ein Tablet mit GSM-Modul ist und Fenster sowie Türmeldern, welche mit der Zentrale kommunizieren. Zu den Schwächen der alten Alarmanlage zählen stetig kaputtwerdende Akkus, eine veraltete Benutzeroberfläche welche nur vor Ort Einstellungsmöglichkeiten bietet, welche nicht alle Bedürfnisse wie das gezielte Steuern einzelner Melder ermöglicht. Weiters ist ein Monitoring der Akkustände der Melder oder andere Auswertungsmöglichkeiten wie Statistiken gar nicht möglich.

Das neue Homesecurity System soll diese Schwächen ausmerzen. Dafür soll eine Smarthome nahe Hardwarelösung z.B. ESP-32 mit ESP-Home/Homeassistant dienen. Weiters soll das Homesecurity System eine Weboberfläche bieten, welche Zugriff für die Verwaltung des Systems sowie Statistische Auswertungen ermöglicht, auch wenn man nicht direkt Vorort ist.

1.2. Überblick

Der Softwareteil umfasst die serverseitige Verarbeitung der Sensordaten, die Persistierung in einer relationalen Datenbank sowie die Bereitstellung einer webbasierten Benutzeroberfläche zur Visualisierung und Steuerung des Systems. Ziel ist die sichere, zuverlässige und nachvollziehbare Verarbeitung der von den IoT-Meldern erfassten Daten.

1.3. Functional Requirements (Software)

M-1: Das System muss dem User die Möglichkeit bieten, das System scharf zu schalten.

M-2: Das System muss dem User die Möglichkeit bieten, den derzeitigen Status (geöffnet, geschlossen, keine Verbindung, ...) eines Melders auszulesen.

M-3: Das System muss im scharf geschalteten Zustand bei Öffnung eines Melders eine Benachrichtigung erzeugen.

M-4: Das System muss dem User die Möglichkeit bieten, den Akkustand der einzelnen Melder auszulesen

M-5: Das System muss dem User die Möglichkeit bieten, Statistiken zu den Status der Melder einzusehen.

M-6: Das System muss Sensordaten vom IoT-Gateway (ESP32-S3) empfangen

M-7: Das System muss die Sensordaten persistieren.

M-8: Das System muss eine Benutzeranmeldung bereitstellen

M-9: Das System muss eine Session-Verwaltung bereitstellen

M-10: Das System muss Benutzerzugriffe protokollieren.

1.3.1. Soll-Kriterien (S)

S-1: Das System sollte dem User die Möglichkeit bieten Akkuverbrauchsstatistiken der Melder einzusehen

1.3.2. Könnte-Kriterien (C)

C-1: Das System könnte dem User die Möglichkeit bieten sich automatisch scharf zu schalten, sobald der Nutzer außer Reichweite des Systems geht

C-2: um die Synchronisation zwischen zwei unabhängigen relationalen Datenbanksystemen zu demonstrieren könnte das System um eine zweite Datenbank erweitert werden

C-3: Die Weboberfläche könnte Statusänderungen ohne manuelles neu laden anzeigen.

1.3.3. Abgrenzungs-Kriterien (W)

W-1: Das System enthält nicht die Möglichkeit Nachrichten außerhalb von HTTPS an den User zu senden

W-2: Das System wird keine Rollenbasierten Zugriffe unterstützen

1.3.4. Muss-Kriterien (M) Non Functional

M-NF-1: Die Kommunikation zwischen Klienten und Server muss über HTTPS laufen

M-NF-2: User Passwörter müssen gehasht und salted gespeichert werden. (bcrypt argon2)

M-NF-3: Datenintegrität: Sensordaten und Alarmereignisse dürfen nach Speicherung **nicht verändert werden**.

1.4. Functional Requirements (Hardware)

1.4.1. Muss-Kriterien (M)

M-H1: Das System muss in der Lage sein, mindestens 2 Melder zu unterstützen und deren Daten zu sammeln

M-H2: Das System muss in der Lage sein, den aktuellen Status eines Melders auslesen zu können

M-H3: Das System muss in der Lage sein den Akkustand des Melders zu messen

M-H4: Im scharf geschalteten Zustand, beim Öffnen eines Melders muss Das System in der Lage sein einen Alarmfall auszulösen

M-H5:

Das System muss jedem Melder ermöglichen die Sensordaten drahtlos an das IoT-Gateway zu liefern.

1.4.2. Soll-Kriterien (S)

S-H1: Das System sollte in der Lage sein eine Sirene im Alarmfall auszulösen

1.4.3. Könnte Kriterien

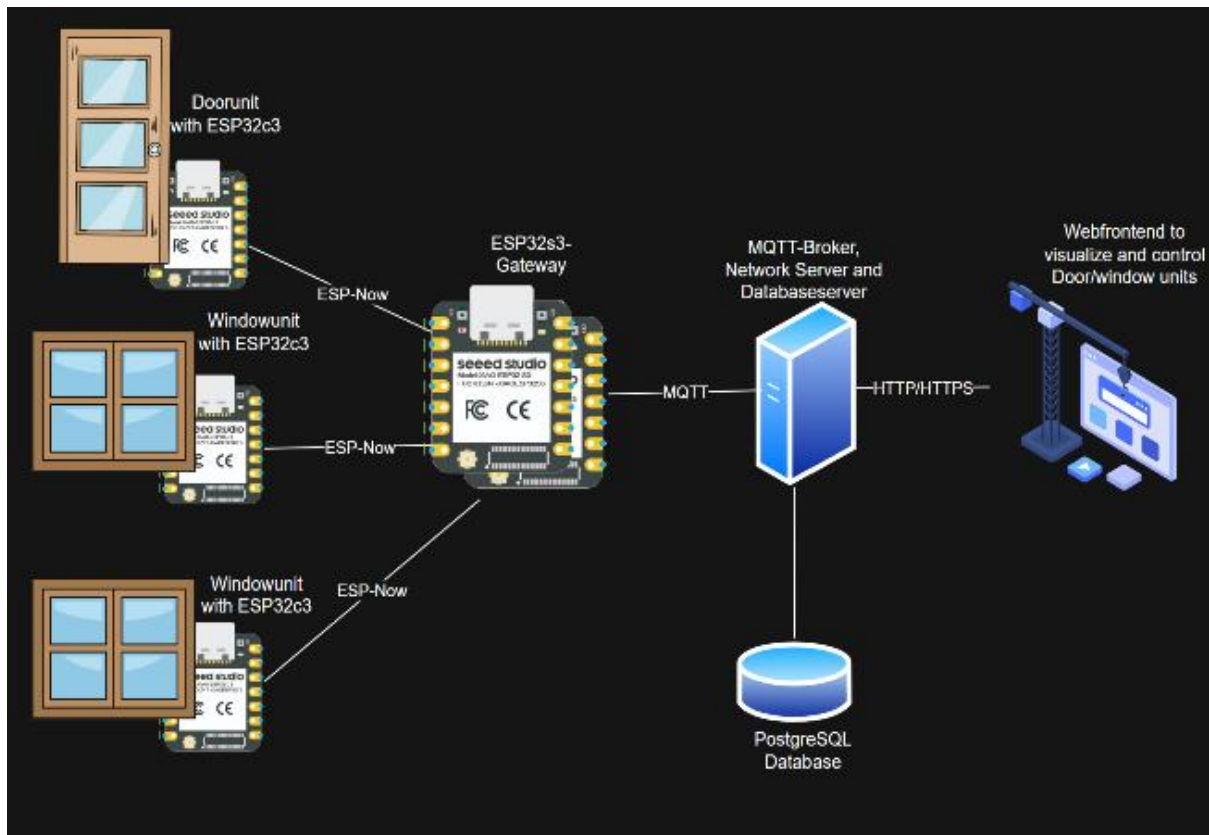
K-H1: Das System könnte bei Verbindungsverlust die Payload erneut übermitteln

1.5. Non-functional Requirements (Hardware)

M-NF-1: Die Akkulaufzeit eines Melders in Normalnutzung muss mindestens 1 Woche betragen

M-NF-2: Die Access Technology muss sinnvoll innerhalb eines Einfamilienhauses einsetzbar sein, d.h. Reichweiten von 10m ohne direkte Sichtlinie und 25m mit direkter Sichtlinie unterstützen (ESP-NOW)

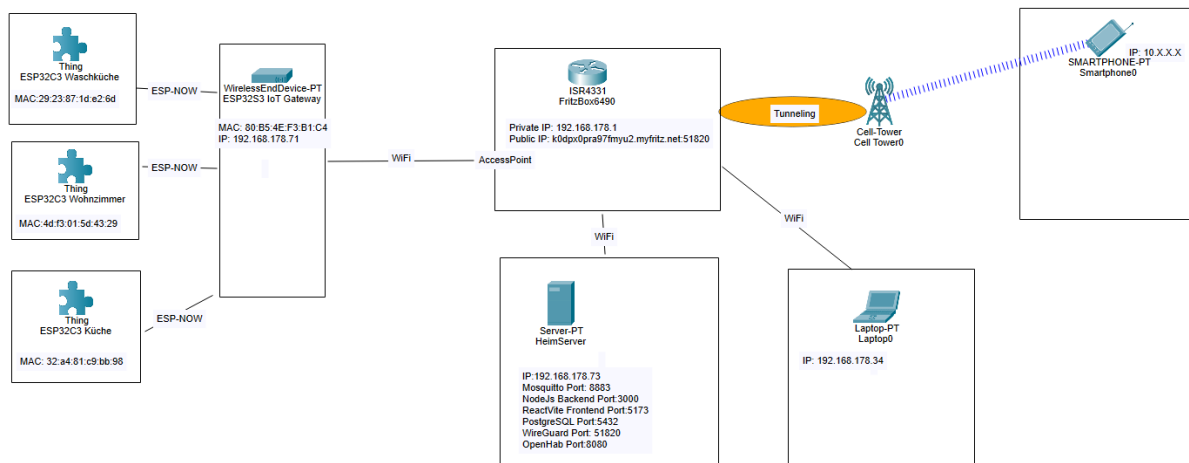
1.6. Technology Stack



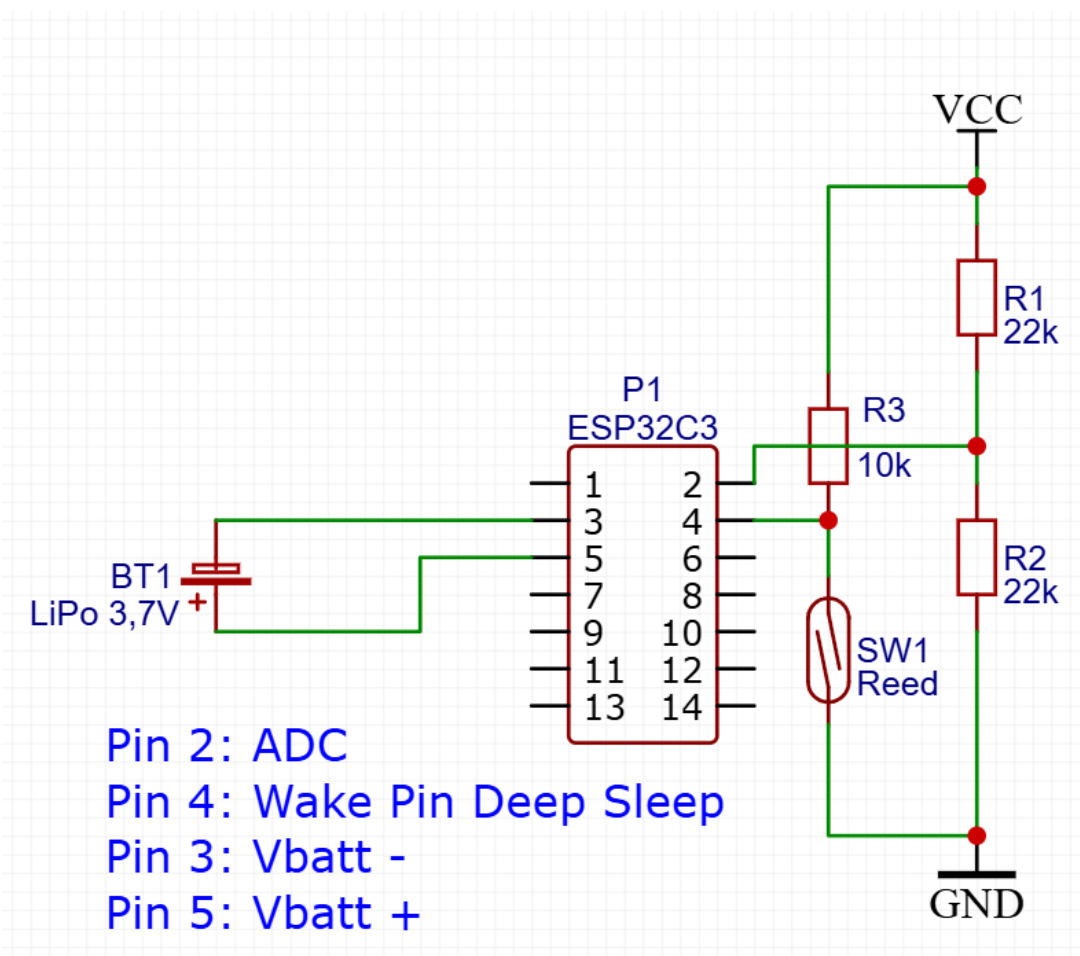
Technology Stack:

- Backend Framework: [Node.js](#), bietet viele Vorteile gegenüber anderen Backendtechnologien wie libraries und Frameworks die gezielt für Webentwicklung entworfen wurden wie z.B. bcrypt, express ...
- Frontend Framework: React vite, bietet viele Funktionalitäten, um responsives Design zu erleichtern und State-Änderungen zu verarbeiten.
- Session Management: Express.js mit JWT, dadurch werden Sessions auf Serverseite validiert.
- Relationale Datenbank(en) mit PostgreSQL, für die strukturierte Datenhaltung eignet sich eine SQL-Datenbank am besten, da hier keine rapiden oder unvorhersehbare Änderungen in der Datenstruktur auftreten werden.
- MQTT-Broker: Mosquitto MQTT, ist ein Leichtgewichtiger und beliebter MQTT-Broker der schnell aufgesetzt ist und mit den richtigen Einstellungen sicher und professionell eingesetzt wird.
- Microcontroller: ESP32. Die ESP32 Reihe ist selten die perfekte Lösung für Serienproduktion, eignet sich jedoch hervorragend für prototyping sowie research and development. Vor allem zeichnen sie sich durch das große Feature Set (Akku-Support mit Ladeschaltung, Analog Digitalwandler, kleiner Formfaktor, starke Rechenleistung, eigenes Low Energy Protokoll ESP-NOW, ...) aus!

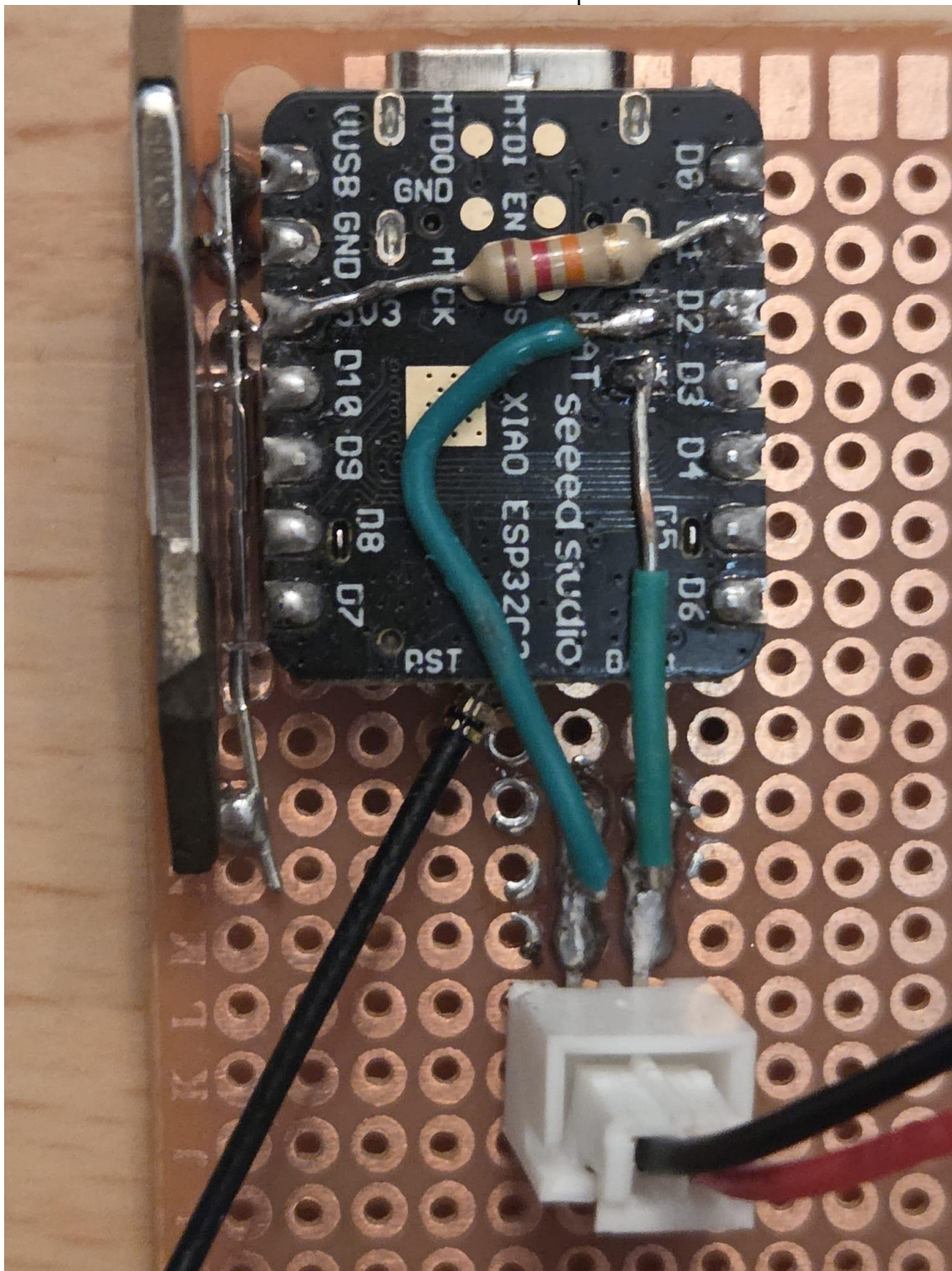
1.7. Netzwerkplan



1.8. Schaltplan



Funktionaler Aufbau eines Türmelders auf Lochrasterplatte:



1.9. Sicherheitsmaßnahmen

Das Node.js Server backend implementiert Session Management via JWT. Es wird eine gültige Session benötigt um auf die REST API zuzugreifen

```
function authenticateToken(req, res, next) {  
  const token = req.cookies.token;  
  
  if (!token) return res.sendStatus(401);  
  
  jwt.verify(token, process.env.JWT_SECRET, (err, user) => {  
    if (err) return res.sendStatus(403);  
  
    req.user = user;  
    next();  
  });  
}
```

Alle Verbindungen des Server backends verwenden https und self-signed mkcert Zertifikate:

```
https.createServer(options, app).listen(3000, '0.0.0.0', () => {  
  console.log("Server running on port 3000");  
  console.log(`Swagger UI available at: https://localhost:3000/api-docs`);  
  console.log(`API JSON available at: https://localhost:3000/swagger.json`);  
});
```

Zertifikats-Viewer: OU=MARLAP\mariu@MarLap (Marius Marik), O=mkcert development certificate

Allgemein

Details

Ausgestellt für

Allgemeiner Name (CN)	<Gehört nicht zum Zertifikat>
Organisation (O)	mkcert development certificate
Organisationseinheit (OU)	MARLAP\mariu@MarLap (Marius Marik)

Ausgestellt von

Allgemeiner Name (CN)	mkcert MARLAP\mariu@MarLap (Marius Marik)
Organisation (O)	mkcert development CA
Organisationseinheit (OU)	MARLAP\mariu@MarLap (Marius Marik)

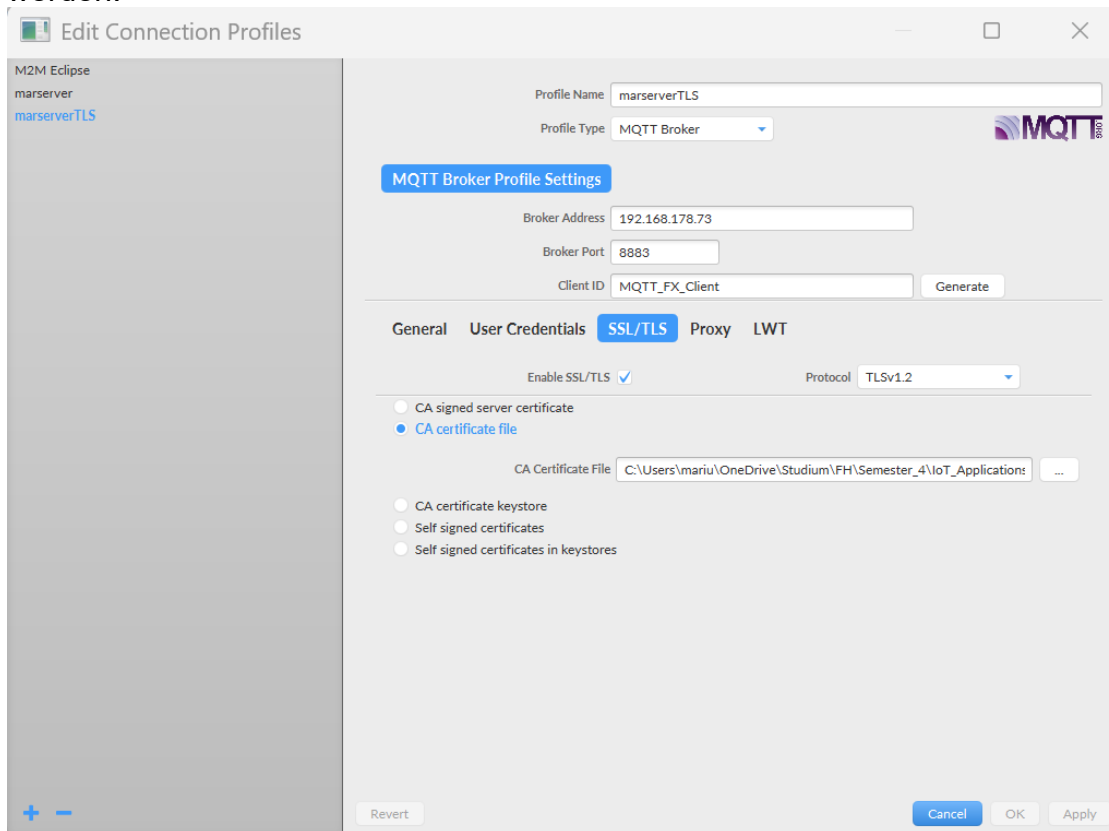
Gültigkeitsdauer

Ausgestellt am	Donnerstag, 28. Mai 2026 um 14:28:56
Gültig bis	Montag, 28. August 2028 um 14:28:56

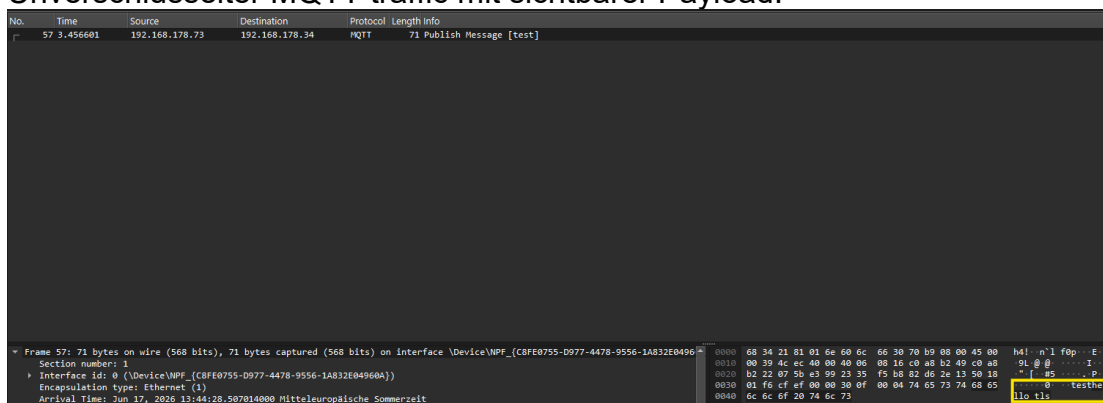
SHA-256-Fingerabdrücke

Zertifikat	abe0f1ca0fc2469ca2b9d3916fdf24cd43dcf6cf5aa94ff280f4e91929940568
Öffentlicher Schlüssel	36068e7ad30420cbe8c30102568bd1794662bdaccbce8d766d58a855493bcebc

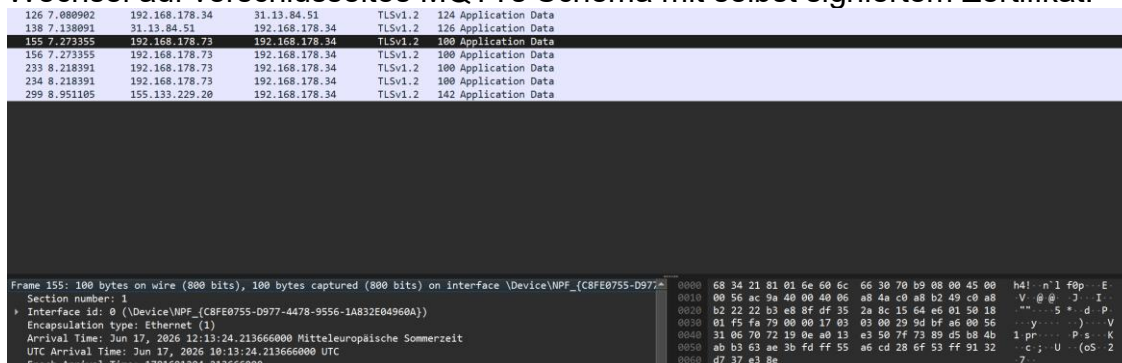
Der Mosquitto MQTT-Broker verwendet TLS mit self-signed Zertifikaten. Die Payload wird verschlüsselt und kann nicht einfach mit Tools wie Wireshark ausgelesen werden.



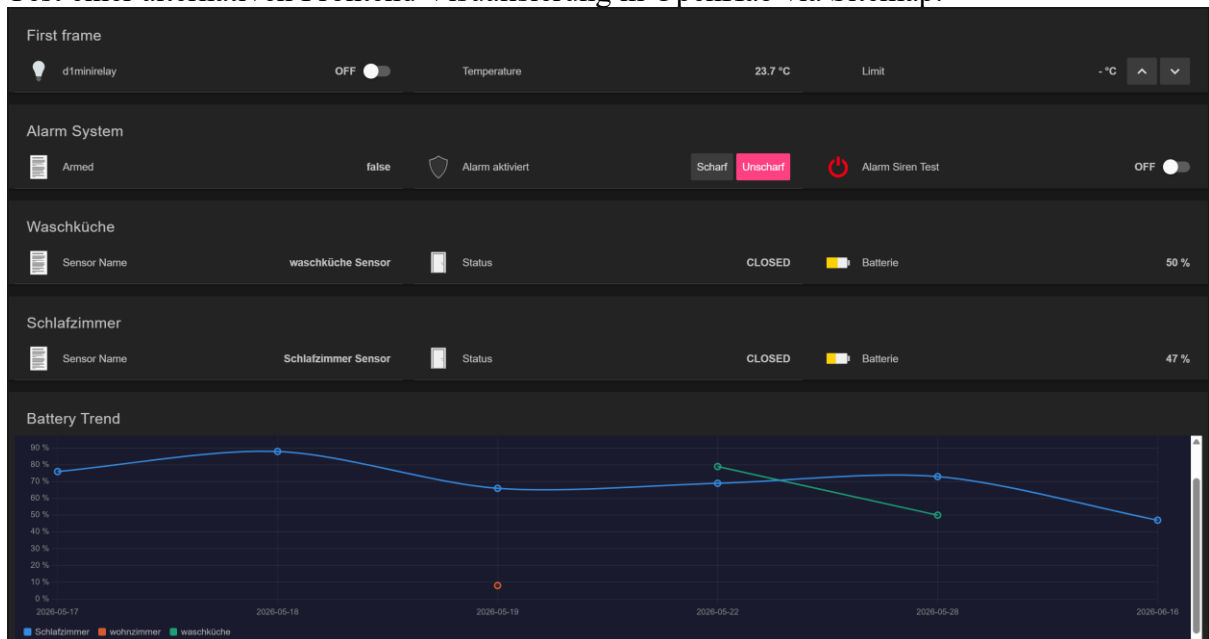
Unverschlüsselter MQTT traffic mit sichtbarer Payload:



Wechsel auf verschlüsseltes MQTTs Schema mit selbst signiertem Zertifikat:



Test einer alternativen Frontend Visualisierung in OpenHab via Sitemap:

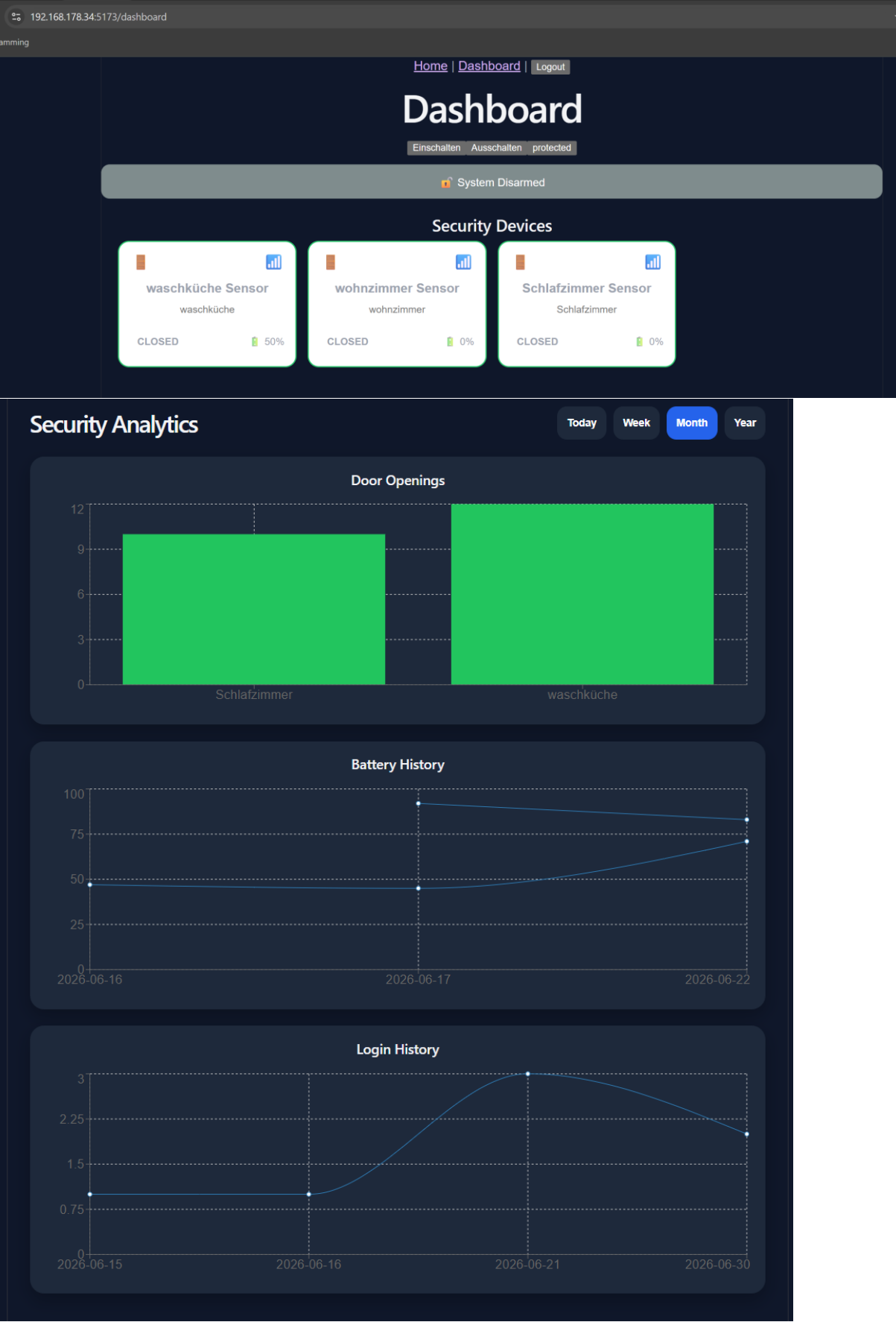


Postgres Database:

The screenshot displays the DBeaver SQL client interface with the following components:

- Top Menu Bar:** File, Edit, Navigate, Search, SQL Editor, Database, Window, Help.
- Database Navigator (Left Panel):** Shows a tree view of the database structure. The 'public' schema is selected, showing tables like 'alarm_state', 'loginlog', 'sensorlog', and 'users'. The 'sensorlog' table is highlighted with a yellow box.
- SQL Editor (Center Panel):** Contains two SQL scripts. The first script, 'createTable.sql', defines a table 'sensorlog' with columns 'topic', 'battery', and 'ts'. The second script, 'Script 4', contains two queries: one to select all data from 'sensorlog' ordered by 'ts desc', and another to select distinct data from 'loginlog' and 'users' based on a time interval.
- Query Results (Bottom Panel):** Displays the results of the first query, showing a table with columns 'id', 'topic', 'state', 'battery', and 'ts'. The results are filtered to show only the first 5 rows.

Custom Frontend:



IoT Gateway Code snippet:

```
IoTGateway | Arduino IDE 2.3.8
File Edit Sketch Tools Help

XIAO_ESP32S3

IoTGateway.ino
99 }
100
101 void mqttSetup()
102 { ...
121 }
122     WiFi.setSleep(false);
123     esp_wifi_set_ps(WIFI_PS_NONE);
124     esp_wifi_set_channel(WiFi.channel(), WIFI_SECOND_CHAN_NONE);
125
126     Serial.print("WiFi status: ");
127     Serial.println(WiFi.status());
128
129     Serial.print("ESP IP: ");
130     Serial.println(WiFi.localIP());
131
132
133     uint8_t primaryChannel;
134     wifi_second_chan_t secondChannel;
135
136     esp_wifi_get_channel(&primaryChannel, &secondChannel);
137     Serial.print("Current WiFi channel: ");
138     Serial.println(primaryChannel);
139
140     mqttClient.loopStart();
141 }
142
143 void setup() {
144     pinMode(sirenPin, OUTPUT);
145     Serial.begin(115200);
146     delay(1000);
147
148     WiFi.mode(WIFI_STA);
149     WiFi.disconnect();
150     printMacAddress();
```

Door/Window -unit Code snippet:

```
int getBatteryLevel()
{
    uint32_t Vbatt = 0;
    for(int i = 0; i < 16; i++) {
        Vbatt = Vbatt + analogReadMilliVolts(A0); // ADC with correction
    }
    float Vbattf = 2 * Vbatt / 16 / 1000.0; // attenuation ratio 1/2, mV --> V
    Serial.println(Vbattf, 3);
    float batteryPercent = voltageToPercent(Vbattf);
    Serial.println(batteryPercent,3);
    return int(batteryPercent);
}

//y(x)=y1+((y2-y1)/(x2-x1))·(x-x1)
float voltageToPercent(float v) {
    if (v >= 4.2) return 100;
    if (v >= 4.0) return 80 + (v - 4.0) * (100 - 80) / (4.2 - 4.0);
    if (v >= 3.8) return 50 + (v - 3.8) * (80 - 50) / (4.0 - 3.8);
    if (v >= 3.6) return 20 + (v - 3.6) * (50 - 20) / (3.8 - 3.6);
    if (v >= 3.3) return (v - 3.3) * (20) / (3.6 - 3.3);
    return 0;
}

void loop(){

    if(digitalRead(wakePin) == LOW)
    {
        sendSensorState(false);
        Serial.println("Going to sleep now");
        esp_deep_sleep_start();
    }

    esp_wifi_get_channel(&primaryChannel, &secondChannel);

    Serial.print("Current WiFi channel: ");
    Serial.println(primaryChannel);

    batteryLevel = getBatteryLevel();
    //send Data to iot Gateway
    sendSensorState(true);
}
```


1.11. Reflexion

Viele Dinge konnten wie geplant umgesetzt werden, doch wie üblich stieß man auf unerwartete Hindernisse in der Umsetzung.

Die größte unerwartete Hürde war das ständige Parsen von Daten, sei es von Daten sei es von Strings zu MQTT-Topics oder von Zeitstempeln zu Datumsformaten oder Teilen davon.

Jedes System arbeitet mit ähnlichen aber selten gleichen Formaten und diese müssen immer wieder gezielt angepasst werden.

```
function formatDate(date) {  
  const d = new Date(date);  
  
  const year = d.getFullYear();  
  const month = String(d.getMonth() + 1).padStart(2, "0");  
  const day = String(d.getDate()).padStart(2, "0");  
  
  const hours = String(d.getHours()).padStart(2, "0");  
  const minutes = String(d.getMinutes()).padStart(2, "0");  
  const seconds = String(d.getSeconds()).padStart(2, "0");  
  
  return `${year}-${month}-${day} ${hours}:${minutes}:${seconds}`;  
}
```

Eine weitere großer Erkenntnis waren Probleme in der Netzwerktechnik wie WLAN-Channels die automatisch vom Router gewechselt werden und zu Verbindungsverlusten zwischen Türmeldern und IOT Gateway führten. Wireguard VPN Sessions, welche z.B. im HCW-Netzwerk blockiert werden und Zugriffe aufs Heimnetzwerk verhinderten.

Generell Verbindungsprobleme, die oft keinen Einfach erkennbaren Grund haben und über mehrere Ebenen hinweg getestet werden müssen.

Das Debugging durch mehrere Ebenen hinweg war auch immer wieder eine Herausforderung: wurden Statusänderungen am Türmelder nicht im Frontend angezeigt begann die Fehlersuche. Oft war Divide and Conquer Fehlersuche der beste Ansatz da die Datenbank genau die Mitte des Systems darstellt und alle Informationen der Hardwareebene beinhaltet und die Grundlage für das Webfrontend ist.

Der Umstieg auf HTTPS bringt immer Herausforderungen mit sich und sollte bestenfalls früh in der Entwicklung umgesetzt werden. In meinem Fall war das einer der letzten Schritte und führte natürlich zu Problemen im Deployment und Testing, da andere Zertifikate für die Testumgebung am Laptop und die Produktivumgebung am Heimserver benötigt wurden.

1.12. Outcome

Im Prinzip wurden alle angestrebten Funktionen umgesetzt bis auf die Könnte Kriterien K-H1, C-1 und C-2. Es ist eine funktionale Alarmanlage umgesetzt worden welche nur noch Kleinigkeiten für eine tatsächliche Verwendung benötigt. Zu den offenen Punkten, um das System tatsächlich in Betrieb zu nehmen zählt einerseits ein PCB für die Tür und Fenstermelder damit der Aufbau einfacher und schneller gemacht werden kann als auf dem Probehafte Lochrasteraufbau. Andererseits wird noch ein Gehäuse benötigt dieses lässt sich jedoch erst mit CAD modellieren und 3D Drucken sobald die Maße des PCB feststehen.

1.13. Zeitaufwand

Arbeitspaket	Tätigkeit	Stunden
1.Projektplanung- und Technologierecherche	Ideenfindung, beschreibung des Mehrwerts, abgrenzung zu existierenden Projekten	4
1.Projektplanung- und Technologierecherche	Erstellen des Netzwerkplanes	3
1.Projektplanung- und Technologierecherche	Konkretisieren der Netzwerktechnologien (Protokolle) und Komponenten (Broker, Webserver)	3
1.Projektplanung- und Technologierecherche	Suche und Bestellung von passenden Hardwarekomponenten	4
2.Schaltungsdesign und Microcontroller Software	Entwerfen der Schaltung für Tür und Fenstermelder	5
2.Schaltungsdesign und Microcontroller Software	Implementieren von Türmelder Software	18
2.Schaltungsdesign und Microcontroller Software	Steckbrettaufbau und Test	3
2.Schaltungsdesign und Microcontroller Software	Prototypen layouting und Aufbau	8
2.Schaltungsdesign und Microcontroller Software	Implementieren von IoTGateway Software	14
2.Schaltungsdesign und Microcontroller Software	Refactoring	3
3. Einrichten des Homeservers	Recherche und Anschaffung Komponenten	2,5
3. Einrichten des Homeservers	Installation und Grundlegendes Setup	6
3. Einrichten des Homeservers	Installation und kennenlernen postgresSQL	4
3. Einrichten des Homeservers	Installation und testing Mosquitto	2
4.Node - Backendentwicklung	Grundlegendes Setup	1,5
4.Node - Backendentwicklung	Session Management mit JWT	2,5
4.Node - Backendentwicklung	Einrichten von MQTT und Datenbankbindung	2,5
4.Node - Backendentwicklung	Erstellen von Endpoints, parsing funktionen und testing	20
4.Node - Backendentwicklung	Entwicklung von wiederverwendbarem .env setup	1
4.Node - Backendentwicklung	Deployment auf von Test auf Produktivumgebung	3
4.Node - Backendentwicklung	Refactoring	4
4.Node - Backendentwicklung	Umstellung auf HTTPS	5
5.Web - Frontendentwicklung	Installation und bekanntmachen mit ReactVite	2
5.Web - Frontendentwicklung	Grundlegendes App und Route design	1,5
5.Web - Frontendentwicklung	Erstellung Dashboard mit SensorCard, SensorModal	5
5.Web - Frontendentwicklung	Kommunikation mit Backend, SessionCookie debugging, erstellen von proxy	2
5.Web - Frontendentwicklung	Erstellung Analyticspage mit recharts	6
5.Web - Frontendentwicklung	Refactoring	3
5.Web - Frontendentwicklung	Design Mockup, Umsetzung, Responsive Design mit mobile first ansatz	6
5.Web - Frontendentwicklung	Erstellung Login, redirection	2

6.Dokumentation	Erstellung Kriterien nach MOSCOW	4
6.Dokumentation	Projekttagbuch, Reflexion, Projektbeschreibung	4
6.Dokumentation	Erstellung von Grafiken und Präsentationsmedien	8
7.Datenbankdesign	Designen und Erstellen von Tabellen mit Constraints	4
7.Datenbankdesign	Schreiben und testen von SQL scripten für Auswertungen	8
7.Datenbankdesign	Erstellen von Views und storedprocedures	3
Gesamt	Gesamt	177,5

1.14. GitHub Repository

<https://github.com/Ibexees/projHomeSec/tree/https>